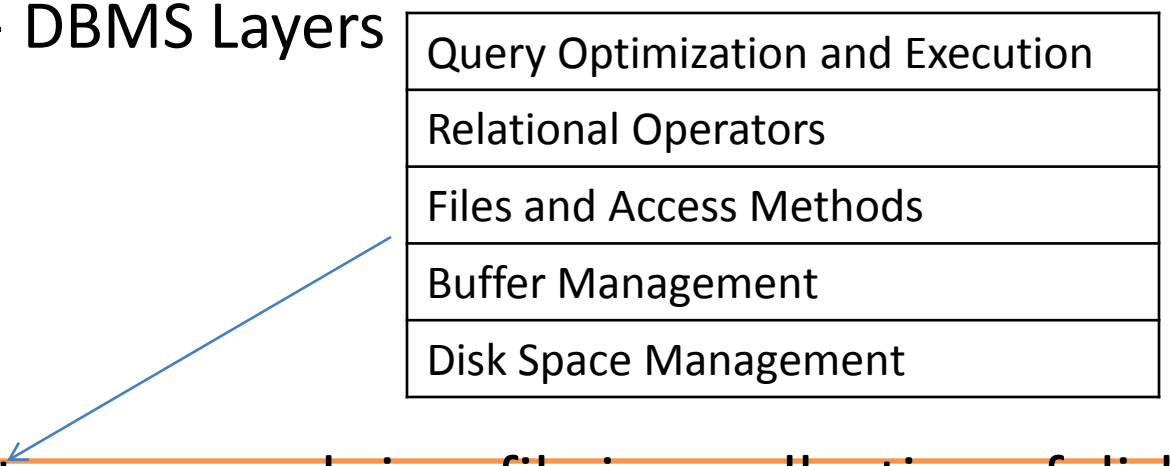
Chapter 8:
File Organization and Indexing

File Organization

- File organization: is a method of arranging the records in a file when the file is stored on disk. A relation is typically stored as a file of records.
  - DBMS Layers



Stores records in a file in a collection of disk pages.
Keeps track of pages allocated to each file.
Tracks available space within pages allocated to the file.

Data on External Storage

- A DBMS stores vast amounts of data and the data has to be preserved across program executions.
- Therefore, data is stored on external storage and fetched into main memory as needed for processing.
- The unit of information that is read and written to a disk is Page.
- Higher layer of DBMS views this pages as unified files and can read or write records to these files.

Classification of Physical Storage Media

Several types of data storage exist in most computer system

These storage are classified by

- Speed with which data can be accessed
- Cost per unit of data
- Reliability
  - data loss on power failure or system crash
  - physical failure of the storage device
- Can differentiate storage into:
  - volatile storage: loses contents when power is switched off
  - non-volatile storage: Contents persist even when power is switched off.

Storage Media

Among the media typically available are these:

- Cache:
  - The fastest and most costly form of storage.
- Main Memory:
  - The general purpose machine instructions operate in main memory
  - used for data that are available to be operated.
  - Is volatile in nature.
- Flash Memory:
  - Is non-volatile in nature
  - Reading data from flash memory is at speed of that of main memory

Storage Media

- Flash Memory:
  - Writing data to flash memory is slower
  - More expensive than disks
  - Used for applications with read workloads that require fast random accesses.
  - Mostly used in embedded systems like in hand held devices and other digital electronic devices.
- Magnetic Disk :
  - Secondary storage, used for long term storage of data.
  - Data should be moved to main memory before in use.
  - It is non volatile in nature.
  - Direct Memory Access.

Storage Media

- Optical Storage:
  - Random access of data
  - Used as secondary access of data for long term .
  - Mostly found in compact disk (CD) and digital video disk(DVD).
  - Data are stored optically on a disk and read by a laser.
  - Capable of storing high data in Gigabytes.
  - Found both in read only and read write form as CD-R, CD-RW,DVD-R,DVD-RW
- Magnetic Tape :
  - Is referred as sequential access storage.
  - Primarily used for backup and archival data .
  - Cheaper than disks but also access to data is slower.

Storage Media

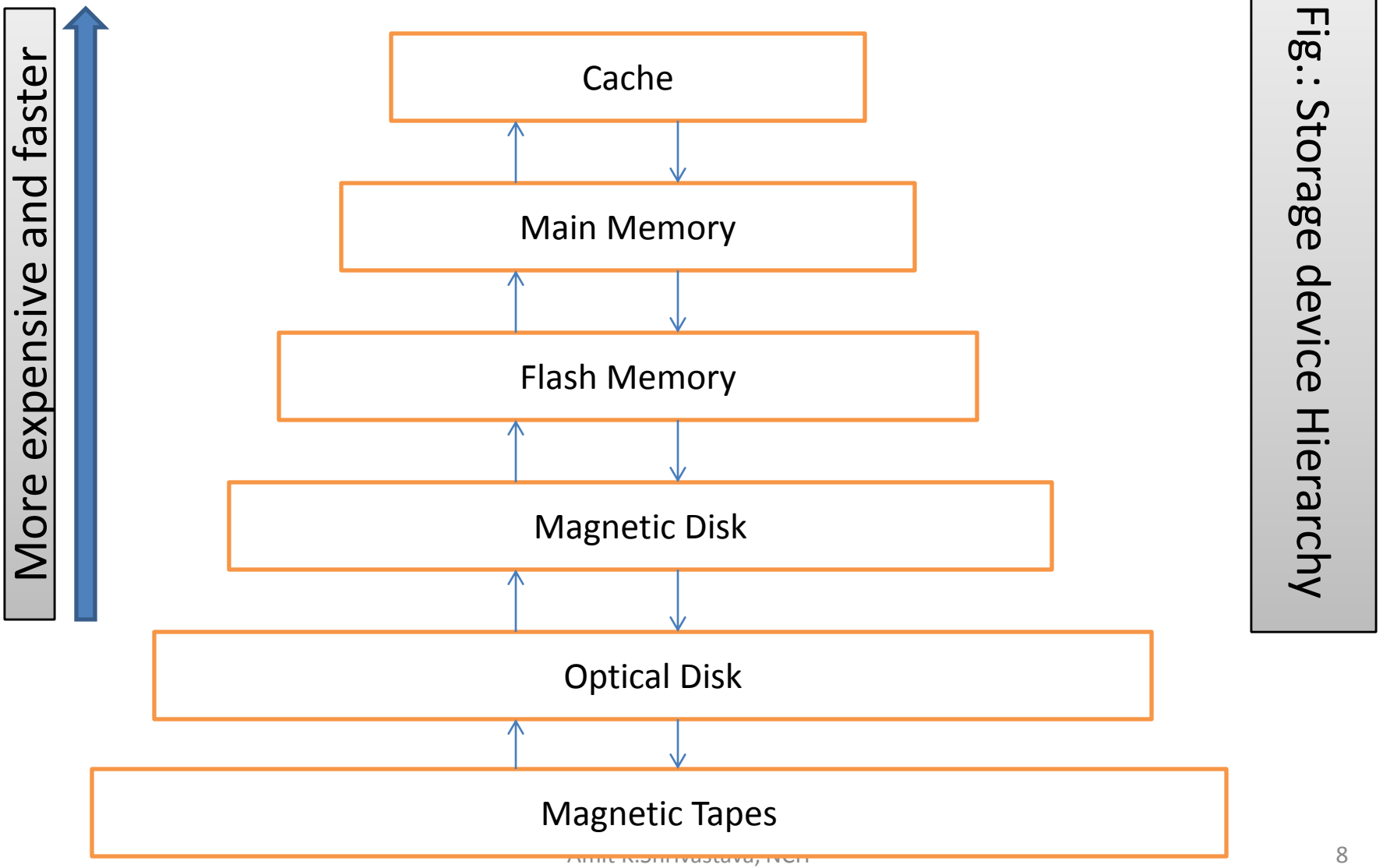


Fig.: Storage device Hierarchy

Buffer Manager

- Subsystem that is responsible for loading pages from external storage to the main memory (buffer pool) when needed.
- File & Index layers make calls to the buffer manager.
- If block is already in buffer, BM passes address of block in main memory.
- Else, BM first allocates space in the buffer for block.
- BM reads in requested block from disk to buffer and passes address of block in main memory.
- Idea is to Keep as many blocks(pages) in memory as possible to reduce disk accesses.
- Replacement Policies
  - LRU(Least-Recently-Used pages … are discarded => the oldest are discarded first)
  - MRU(Most-Recently-Used, the newest are discarded first)
  - LFU(Least-Frequently-Used)

File Organization

- Method for arranging a collection of records and supporting the concept of a file.
- A file is organized logically as a sequence of records. These records are mapped into the disk blocks.
- Each file is also logically partitioned into fixed length storage units called blocks, which are the units of both storage allocation and data transfer.
- Normally the block size range from 4 to 8 kilo bytes by default to Giga Bytes.
- Blocks contains records that is determined by the form of physical data organization being used.
- The size of the records should not be greater than block considering the fact that the field might be of image and it can varies in size dramatically.
- The another fact is that the entire record should be contained in one block rather than divided into separate blocks. This will benefits in speeding up the data access.
- Records may varies in size in one particular file system, so it is to be addressed.

Fixed Length Records

- Giving the attributes fixed size for an entity in terms of byte.
- For the file student, record may contain id int, name varchar(20), age int. For each record the space required is 24 bytes.
- **Problems**
  - The file records are of the same record type, but one or more of the fields are of varying size.
  - The file records are of the same record type but one or more of the fields may have multiple values for individual's records.
  - The file records are of the same records type but one or more of the fields are optional.
  - But it is very hard to occupy the block size of multiple of 24 bytes. So, it may be possible that when the block size obtain its max limit, it might be possible that the information of records may shift to another block.
  - While deleting the records from file, it should be either marked as deleted or should be occupied by next new record.

Fixed Length Records

- **Solutions**
  - The records should be entered into the block after the block size computation (dividing block size with the record size), by leaving the remaining block size unused.
  - While deleting record, we could move the record that came after it into the space formerly occupied by the deleted record and so on, until every record following the deleted record has been moved ahead. Or it might be easier to move the final record of the file into the space occupied by the deleted record.
  - The file header is used for storing the information of deleted records as well as available records. While inserting the new record, if the file contains the deleted spaces it will insert the record in that position and so on, if it does not contain any deleted position the record is placed at the end of the file

Fixed Length Records

Record 0	A-102	Kalimati	4000
Record 1	A-201	Patan	5000
Record 2	A-302	Bhaktapur	6000
Record 3	A-402	kalanki	4000
Record 4	A-103	kalimati	5000
Record 5	A-502	kirtipur	3000
Record 6	A-105	kalimati	5000
Record 7	A-202	patan	4000
Record 8	A-403	kalanki	4000

Record 0	A-102	Kalimati	4000
Record 1	A-201	Patan	5000
Record 3	A-402	kalanki	4000
Record 4	A-103	kalimati	5000
Record 5	A-502	kirtipur	3000
Record 6	A-105	kalimati	5000
Record 7	A-202	patan	4000
Record 8	A-403	kalanki	4000

Record 2 deleted and all records moved up

Fixed Length Records

Record 0	A-102	Kalimati	4000
Record 1	A-201	Patan	5000
Record 8	A-403	kalanki	4000
Record 3	A-402	kalanki	4000
Record 4	A-103	kalimati	5000
Record 5	A-502	kirtipur	3000
Record 6	A-105	kalimati	5000
Record 7	A-202	patan	4000

Record 2 deleted and final record moved

header			
Record 0	A-102	Kalimati	4000
Record 1			
Record 2	A-302	Bhaktapur	6000
Record 3	A-402	kalanki	4000
Record 4			
Record 5	A-502	kirtipur	3000
Record 6			
Record 7	A-202	patan	4000
Record 8	A-403	kalanki	4000

Free list after deletion of records 1,4,6

Variable Length Records

- It is used for
  - storing multiple record type in a file
  - record type that allow variable lengths for one or more fields
  - record types that allow repeating fields, such as arrays.
- Byte String Representation:
  - Attach a special *end of record* (⊥) symbol to the end of each record.
  - Store each record as a string of consecutive bytes.
  - Disadvantages:
    - Not easy to reuse space occupied formerly by a deleted record.
    - No space for records to grow longer.

Variable Length Records

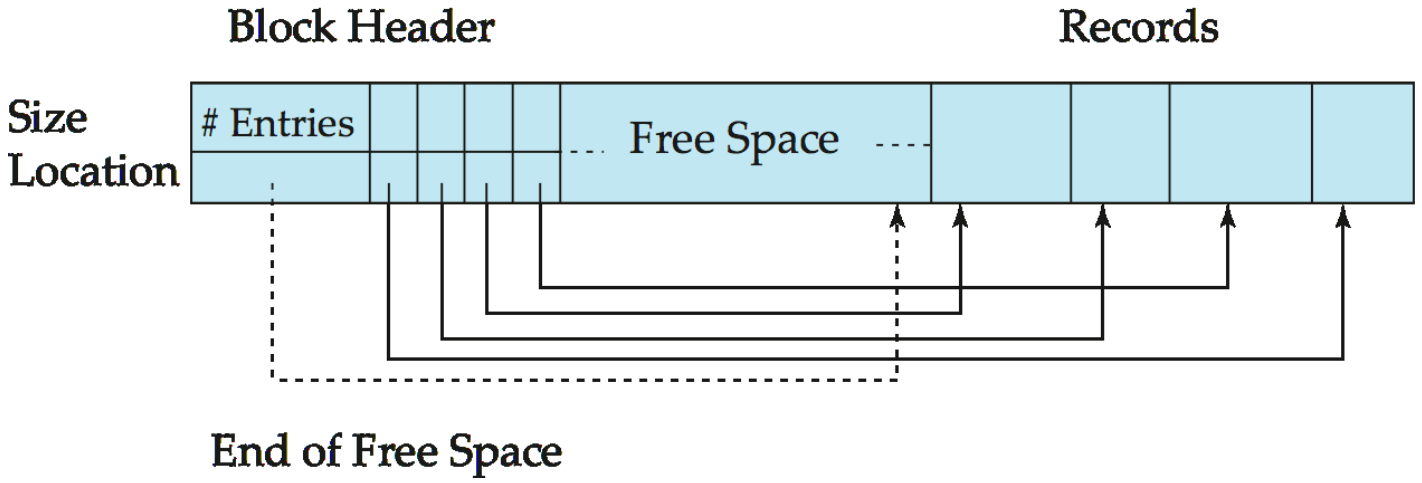
- Byte String Representation:

0	Kalimati	A-102	4000	A-103	5000	A-105	5000	
1	Patan	A-201	5000	A-202	4000			
2	Bhaktapur	A-303	6000					
3	Kalanki	A-402	4000	A-403	4000			
4	Kirtipur	A-502	3000					

- Slotted page structure:
  - Header consists of
    - number of record entries
    - end of free space in the block
    - location and size of each record

Variable Length Records

- Slotted page structure:
  - Records can be moved around within a page to keep them contiguous with no empty space between them; entry in the header must be updated.
  - Pointers should not point directly to record — instead they should point to the entry for the record in header.



Variable Length Records

- Fixed Length Representation:
  - Two ways
    - Reserved Space
    - List Representation

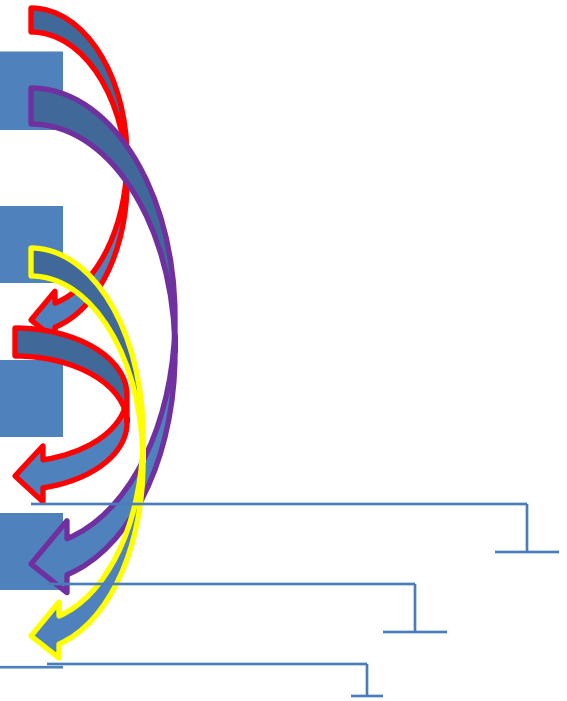
0	Kalimati	A-102	4000	A-103	5000	A-105	5000
1	Patan	A-201	5000	A-202	4000		
2	Bhaktapur	A-303	6000	⊥	⊥	⊥	⊥
3	Kalanki	A-402	4000	A-403	4000		
4	Kirtipur	A-502	3000	⊥	⊥	⊥	⊥

Reserved Space Method

Variable Length Records

- Fixed Length Representation:

Record 0	Kalimati	A-102	4000
Record 1	Patan	A-201	5000
Record 2	Bhaktapur	A-302	6000
Record 3	kalanki	A-402	4000
Record 4		A-103	5000
Record 5	kirtipur	A-502	3000
Record 6		A-105	5000
Record 7		A-202	4000
Record 8		A-403	4000



Linked List Method

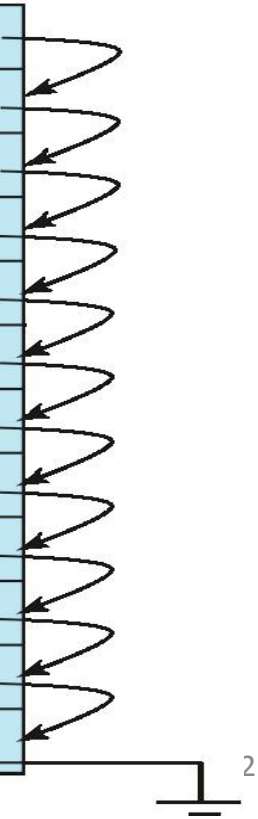
Organization of Records in Files

- **Heap**
  - a record can be placed anywhere in the file where there is space
- **Sequential**
  - store records in sequential order, based on the value of the search key of each record
- **Hashing**
  - a hash function computed on some attribute of each record; the result specifies in which block of the file the record should be placed
- Records of each relation may be stored in a separate file. In a **multitable clustering file organization** records of several different relations can be stored in the same file
  - Motivation: store related records on the same block to minimize I/O

Sequential File Organization

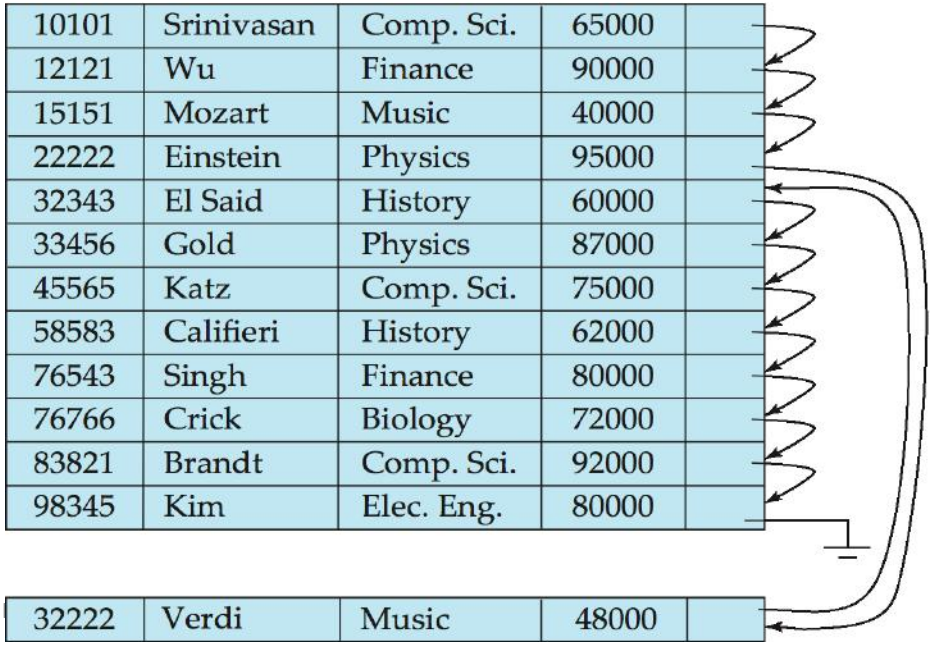
- Suitable for applications that require sequential processing of the entire file
- The records in the file are ordered by a search-key

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



Sequential File Organization

- Deletion – use pointer chains
- Insertion –locate the position where the record is to be inserted
  - if there is free space insert there
  - if no free space, insert the record in an overflow block
  - In either case, pointer chain must be updated
- Need to reorganize the file from time to time to restore sequential order



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

32222	Verdi	Music	48000	

Data Dictionary Storage

- The **Data dictionary** (also called **system catalog**) stores **metadata**; that is, data about data, such as

I. Information about relations
   - names of relations
   - names, types and lengths of attributes of each relation
   - names and definitions of views
   - integrity constraints

II. User and accounting information, including passwords

III. Statistical and descriptive data
   - number of tuples in each relation

IV. Physical file organization information
   - How relation is stored (sequential/hash/…)
   - Physical location of relation

V. Information about indices

Indexing

- Indexing mechanisms are used to speed up access to desired data.
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form



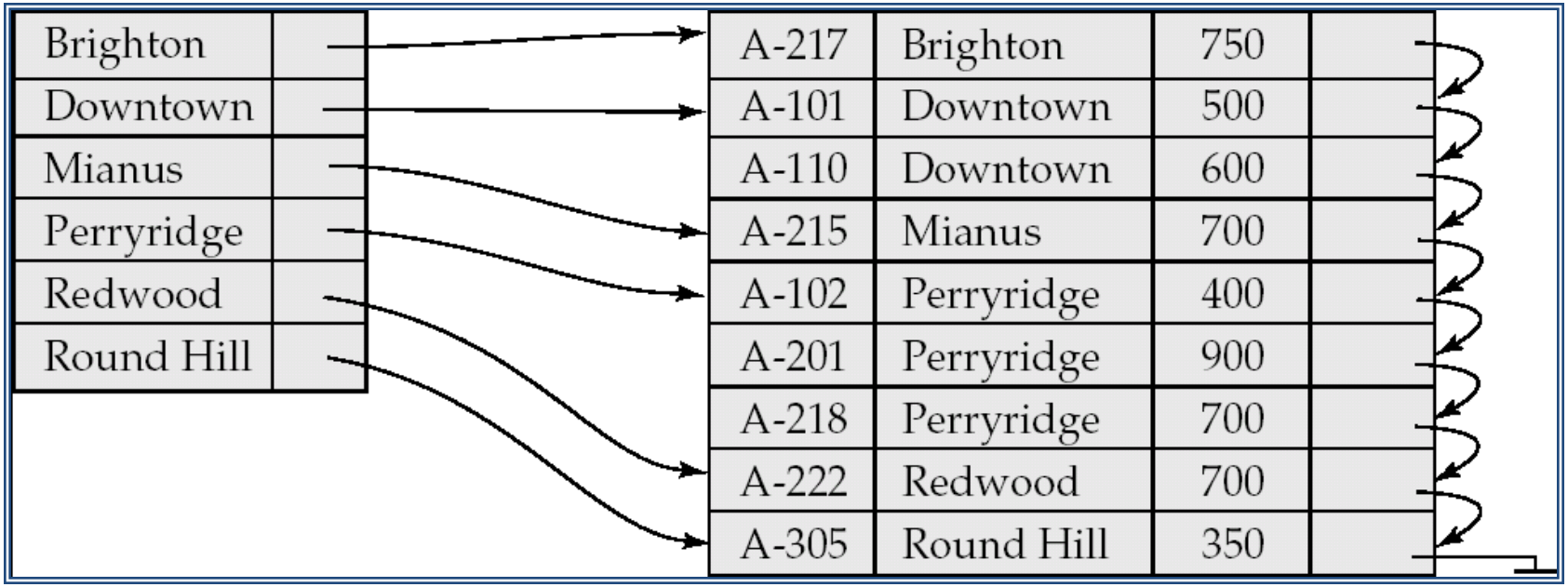
- Index files are typically much smaller than the original file
- Two basic kinds of indices:
  - **Ordered indices:** search keys are stored in sorted order
  - **Hash indices:** search keys are distributed uniformly across "buckets" using a "hash function".
- Index Evaluation Metrics:
  - Access time
  - Insertion time
  - Deletion time
  - Space overhead
  - Access types supported efficiently. E.g.,
    - records with a specified value in the attribute
    - or records with an attribute value falling in a specified range of values.
    - This strongly influences the choice of index, and depends on usage.

1.Ordered Indices

- In an **ordered index,** index entries are stored sorted on the search key value. E.g., author catalog in library.
- **Primary index:** *in a sequentially ordered file*, the index whose search key specifies the sequential order of the file.
  - Also called **clustering index**
  - The search key of a primary index is usually but not necessarily the primary key.
- **Secondary index**: an index whose search key specifies an order different from the sequential order of the file.
  - Also called non-clustering index**.**
- *Index-sequential file: ordered sequential file with a primary index.*

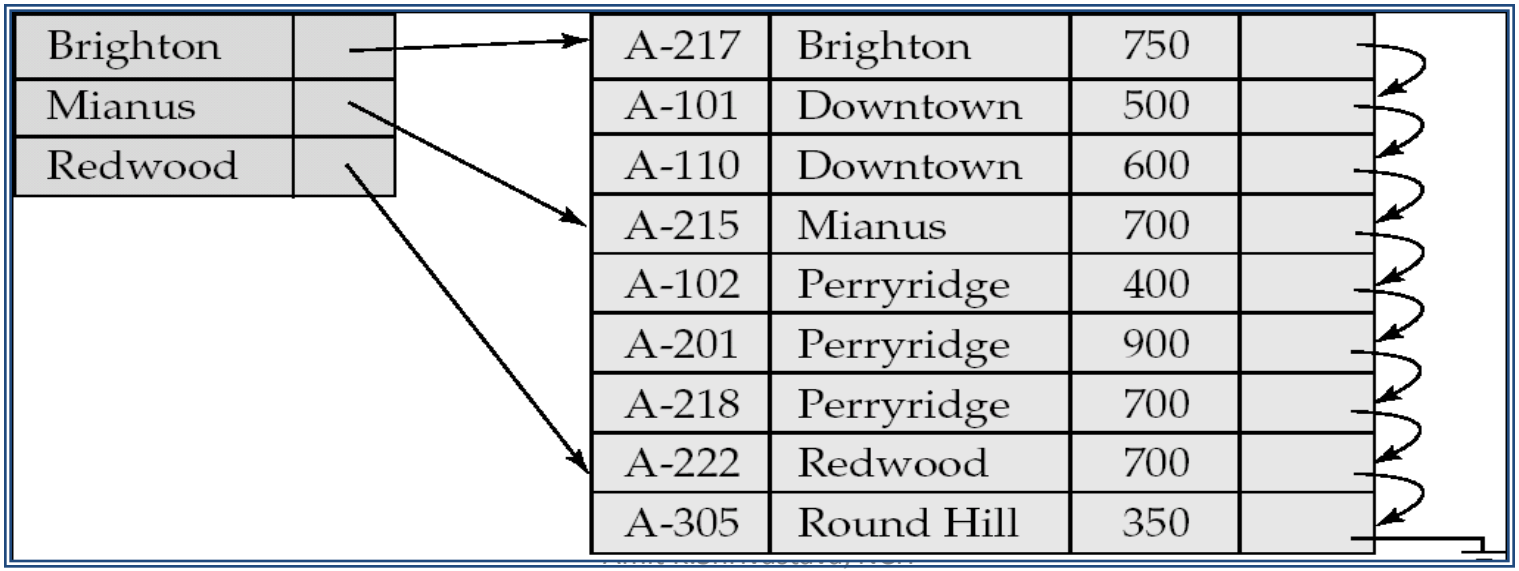
1.1.1Dense index

- Index record appears for every search-key value in the file.
- In dense primary index, the index record contains the search key value and a pointer to the first data record with that search key value.



1.1.2 Sparse Index

- Index records appears for only some search-key values.
- Here also the index record contains the search key value and a pointer to the first data record with that search key value.
- Only applicable when records are sequentially ordered on search-key
- To locate a record with search-key value *K* we:
  - Find index record with largest search-key value < *K*
  - Search file sequentially starting at the record to which the index record points



1.1.3 Multilevel Indices

- If primary index does not fit in memory, access becomes expensive.
- Solution:
  - treat primary index kept on disk as a sequential file and construct a sparse index on it.
    - outer index – a sparse index of primary index
    - inner index – the primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.
- To locate a record, we first use binary search on the outer index to find the record for the largest search-key value less than or equal to the one that we desire.
- The pointer points to the block of inner index. We scan this block until we find the record that has the largest search key value less than or equal to the one that we desire. The pointer in this record points to the block of the file that contains the record for which we are looking

1.1.4 Index Update

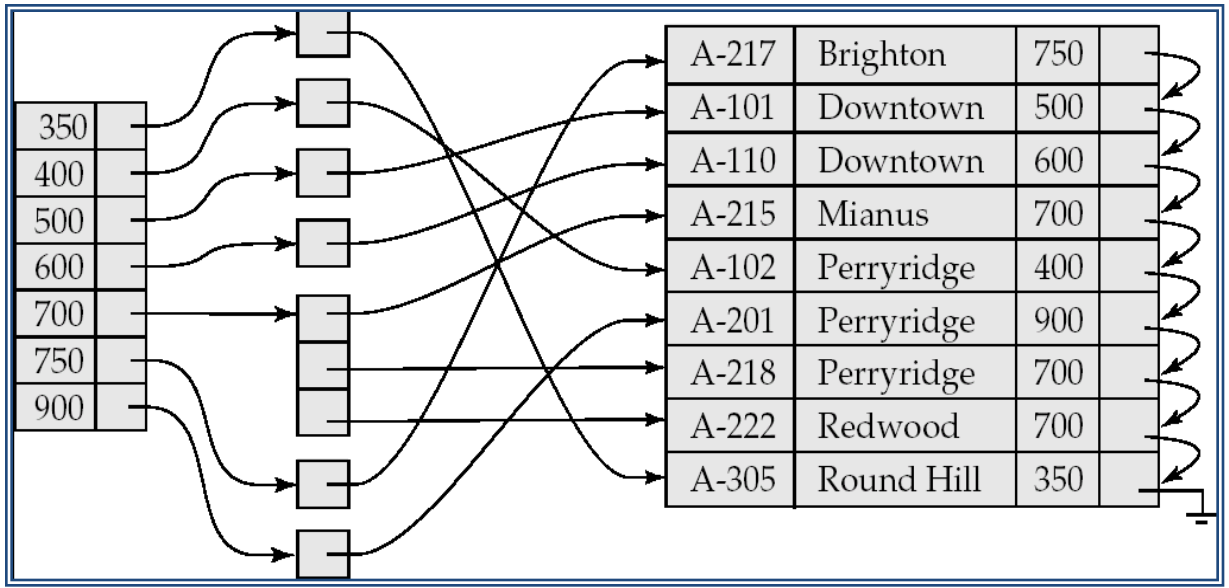
- **Deletion**
  - If deleted record was the only record in the file with its particular search-key value, the search-key is deleted from the index also.
  - Single-level index deletion:
    - **Dense indices** – deletion of search-key: similar to file record deletion.
    - **Sparse indices** –
      - if an entry for the search key exists in the index, it is deleted by replacing the entry in the index with the next search-key value in the file (in search-key order).
      - If the next search-key value already has an index entry, the entry is deleted instead of being replaced.

1.1.4 Index Update

- **Insertion**
  - Single-level index insertion:
    - Perform a lookup using the search-key value appearing in the record to be inserted.
    - **Dense indices** – if the search-key value does not appear in the index, insert it.
    - **Sparse indices** –
      - if index stores an entry for each block of the file, no change needs to be made to the index unless a new block is created.
      - If a new block is created, the first search-key value appearing in the new block is inserted into the index.
  - Multilevel insertion (as well as deletion) algorithms are simple extensions of the single-level algorithms

1.2 Secondary Indices

- Frequently, one wants to find all the records whose values in a certain field (which is not the search-key of the primary index) satisfy some condition.
  - Example 1: In the *account* relation stored sequentially by account number, we may want to find all accounts in a particular branch
  - Example 2: as above, but where we want to find all accounts with a specified balance or range of balances
- We can have a secondary index with an index record for each search-key value
- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be dense, since the file is not sorted by the search key.



1.2.1 Secondary and Primary Indices

- Indices offer substantial benefits when searching for records, but updating indices imposes overhead on database modification - when a file is modified, every index on the file must be updated,
- Sequential scan using primary index is efficient, but a sequential scan using a secondary index is expensive
  - Each record access may fetch a new block from disk
  - Block fetch requires about 5 to 10 micro seconds, versus about 100 nanoseconds for memory access

B^+-Tree Index Files

- B^+-tree indices are an alternative to indexed-sequential files.
- Disadvantage of indexed-sequential files
  - performance degrades as file grows, since many overflow blocks get created.
  - Periodic reorganization of entire file is required.
- Advantage of B^+-tree index files:
  - automatically reorganizes itself with small, local, changes, in the face of insertions and deletions.
  - Reorganization of entire file is not required to maintain performance.
- (Minor) disadvantage of B^+-trees:
  - extra insertion and deletion overhead, space overhead.
- Advantages of B^+-trees outweigh disadvantages
  - B^+-trees are used extensively
- Typical node



  - K_i are the search-key values
  - P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

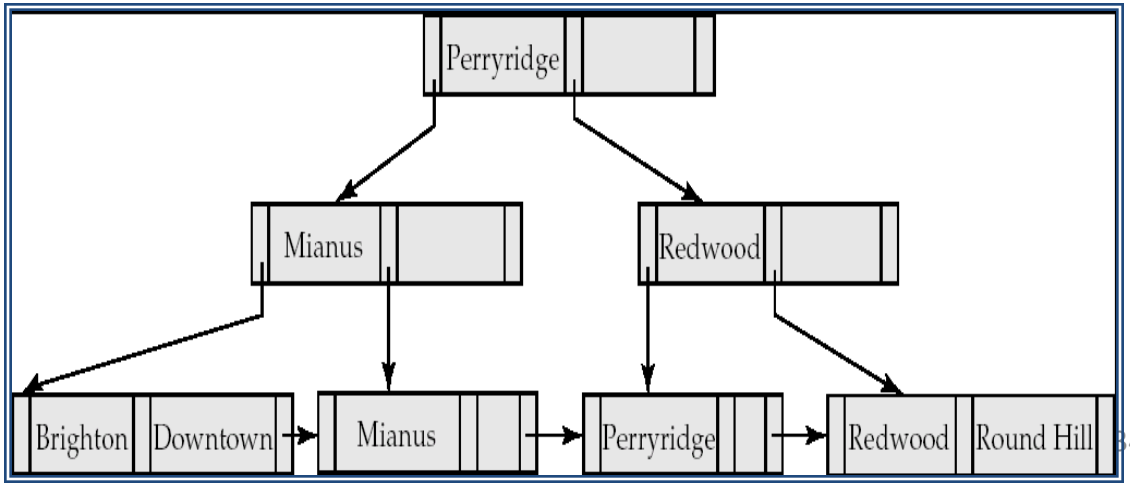
$$K_1 < K_2 < K_3 < \ldots < K_{n-1}$$

- Usually the size of a node is that of a block

B^+-Tree Index Files

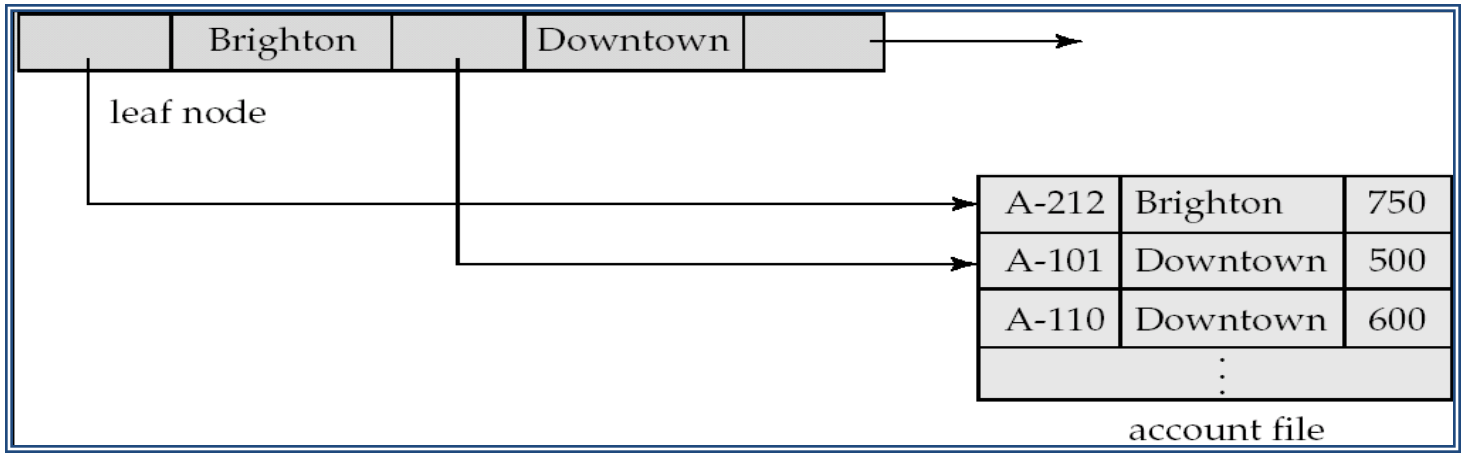
- A B^+-tree is a rooted tree satisfying the following properties:
  - All paths from root to leaf are of the same length
  - Each node that is not a root or a leaf has between $\lceil n/2 \rceil$ and n children.
  - A leaf node has between $\lceil (n–1)/2 \rceil$ and $n–1$ values
  - Special cases:
    - If the root is not a leaf, it has at least 2 children.
    - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n–1)$ values.
  - All values must be at leaf node.

B^+-tree for *account* file ($n = 3$)



Leaf Nodes in B^+-Trees

- Properties of a leaf node:
  - For $i = 1, 2, \ldots, n-1$, pointer P_i either points to a file record with search-key value K_i, or to a bucket of pointers to file records, each record having search-key value K_i. Only need bucket structure if search-key does not form a primary key.
  - If L_i, L_j are leaf nodes and $i < j$, L_i's search-key values are less than L_j's search-key values
  - P_n points to next leaf node in search-key order



Non-Leaf Nodes in B$^+$-Trees

- Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with *m* pointers:
  - All the search-keys in the subtree to which P_1 points are less than K_1
  - For $2 \leq i \leq n - 1$, all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_i
  - All the search-keys in the subtree to which P_n points have values greater than or equal to K_{n-1}
- *Leaf nodes must have between 2 and 4 values ($\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 5$).*
- *Non-leaf nodes other than root must have between 3 and 5 children ($\lceil n/2 \rceil$ and n with $n = 5$).*
- *Root must have at least 2 children.*

B^+-Trees

- Since the inter-node connections are done by pointers, “logically” close blocks need not be “physically” close.
- The non-leaf levels of the B^+-tree form a hierarchy of sparse indices.
- The B^+-tree contains a relatively small number of levels
  - Level below root has at least $2* \lceil n/2 \rceil$ values
  - Next level has at least $2* \lceil n/2 \rceil * \lceil n/2 \rceil$ values
  - .. etc.
  - If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
  - thus searches can be conducted efficiently.
- Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time (as we shall see some details, and more in the book).

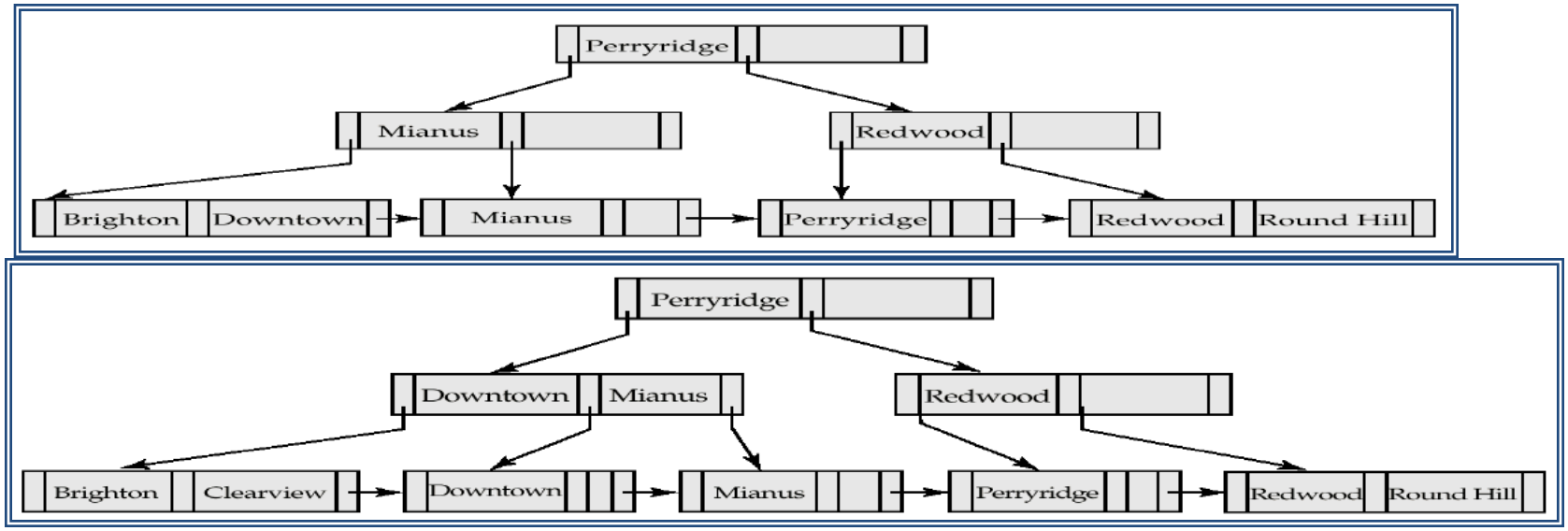
Queries on B^+-Trees

- Find all records with a search-key value of *k.*
  1. *N=root*
  2. Repeat
     1. Examine *N* for the smallest search-key value > *k.*
     2. If such a value exists, assume it is K_i. Then set $N = P_i$
     3. Otherwise $k \geq K_{n-1}$. Set $N = P_n$

     Until *N* is a leaf node
  3. If for some *i*, key $K_i = k$ follow pointer P_i to the desired record or bucket.
  4. Else no record with search-key value *k* exists.
- If there are *K* search-key values in the file, the height of the tree is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.
- A node is generally the same size as a disk block, typically 4Kbytes
  - *n* is typically around 100 (40 bytes per index entry).
- With 1 million search key values and *n* = 100
  - at most $log_{50}(1{,}000{,}000) = 4$ nodes are accessed in a lookup.
    - I.e. at most 4 accesses to disk blocks are needed
- Contrast this with a balanced binary tree with 1 million search key values — around 20 nodes are accessed in a lookup
  - above difference is significant since every node access may need a disk I/O, costing around 20 milliseconds

Updates on B+-Trees: Insertion

1. Find the leaf node in which the search-key value would appear
2. If the search-key value is already present in the leaf node
   1. Add record to the file
   2. If necessary add a pointer to the bucket.
3. If the search-key value is not present, then
   1. add the record to the main file (and create a bucket if necessary)
   2. If there is room in the leaf node, insert (key-value, pointer) pair in the leaf node
   3. Otherwise, split the node (along with the new (key-value, pointer) entry)



B+-Tree before and after insertion of "Clearview"

Updates on B$^+$-Trees: Insertion

- Splitting a leaf node:
  - take the *n* (search-key value, pointer) pairs (including the one being inserted) in sorted order. Place the first $\lceil n/2 \rceil$ in the original node, and the rest in a new node.
  - let the new node be *p,* and let *k* be the least key value in *p.* Insert (*k,p*) in the parent of the node being split.
  - If the parent is full, split it and propagate the split further up.
- Splitting of nodes proceeds upwards till a node that is not full is found.
  - In the worst case the root node may be split increasing the height of the tree by 1.
- Splitting a non-leaf node: when inserting (k,p) into an already full internal node N
  - Copy N to an in-memory area M with space for n+1 pointers and n keys
  - Insert (k,p) into M
  - Copy $P_1, K_1, \ldots, K_{\lceil n/2 \rceil - 1}, P_{\lceil n/2 \rceil}$ from M back into node N
  - Copy $P_{\lceil n/2 \rceil + 1}, K_{\lceil n/2 \rceil + 1}, \ldots, K_n, P_{n+1}$ from M into newly allocated node N'
  - Insert $(K_{\lceil n/2 \rceil}, N')$ into parent N

Updates on B^+-Trees: Deletion

- Find the record to be deleted, and remove it from the main file and from the bucket (if present)
- Remove (search-key value, pointer) from the leaf node if there is no bucket or if the bucket has become empty
- If the node has too few entries due to the removal, and the entries in the node and a sibling fit into a single node, then ***merge siblings:***
  - Insert all the search-key values in the two nodes into a single node (the one on the left), and delete the other node.
  - Delete the pair (K_{i-1}, P_i), where P_i is the pointer to the deleted node, from its parent, recursively using the above procedure.
- Otherwise, if the node has too few entries due to the removal, but the entries in the node and a sibling do not fit into a single node, then **redistribute pointers**:
  - Redistribute the pointers between the node and a sibling such that both have more than the minimum number of entries.
  - Update the corresponding search-key value in the parent of the node.
- The node deletions may cascade upwards till a node which has $\lceil n/2 \rceil$ or more pointers is found.
- If the root node has only one pointer after deletion, it is deleted and the sole child becomes the root.

B-Tree Index Files

- Similar to B+-tree, but B-tree allows search-key values to appear only once; eliminates redundant storage of search keys.
- Search keys in nonleaf nodes appear nowhere else in the B-tree; an additional pointer field for each search key in a nonleaf node must be included.
- Advantages of B-Tree indices:
  - May use less tree nodes than a corresponding B^+-Tree.
  - Sometimes possible to find search-key value before reaching leaf node.
- Disadvantages of B-Tree indices:
  - Only small fraction of all search-key values are found early
  - Non-leaf nodes are larger, so fan-out is reduced. Thus, B-Trees typically have greater depth than corresponding B^+-Tree
  - Insertion and deletion more complicated than in B^+-Trees
  - Implementation is harder than B^+-Trees.
  - Not possible to sequentially scan a table by just looking at leafs.
- Typically, advantages of B-Trees do not out weigh disadvantages.
  - In DBMSs B+-Trees are favored.

Hashing

- Hashing is an effective technique to calculate direct location of data record on the disk without using index structure.
- It uses a function, called hash function and generates address when called with search key as parameters. Hash function computes the location of desired data on the disk.
- Hash Organization:
  - **Bucket:** Hash file stores data in bucket format. Bucket is considered a unit of storage. Bucket typically stores one complete disk block, which in turn can store one or more records.
  - **Hash Function:** A hash function h, is a mapping function that maps all set of search-keys K to the address where actual records are placed. It is a function from search keys to bucket addresses.
    - Choose hash function that assign search key values to buckets in such a way that distribution is either uniform or random.

Static Hashing

- In static hashing, when a search-key value is provided the hash function always computes the same address.
- For example, if mod-4 hash function is used then it shall generate only 5 values.
- The output address shall always be same for that function. The numbers of buckets provided remain same at all times.
- Operation:
  - **Insertion:** When a record is required to be entered using static hash, the hash function h, computes the bucket address for search key K, where the record will be stored.
    - Bucket address = h(K)
  - **Search:** When a record needs to be retrieved the same hash function can be used to retrieve the address of bucket where the data is stored.
  - **Delete:** This is simply search followed by deletion operation.

Bucket Overflows

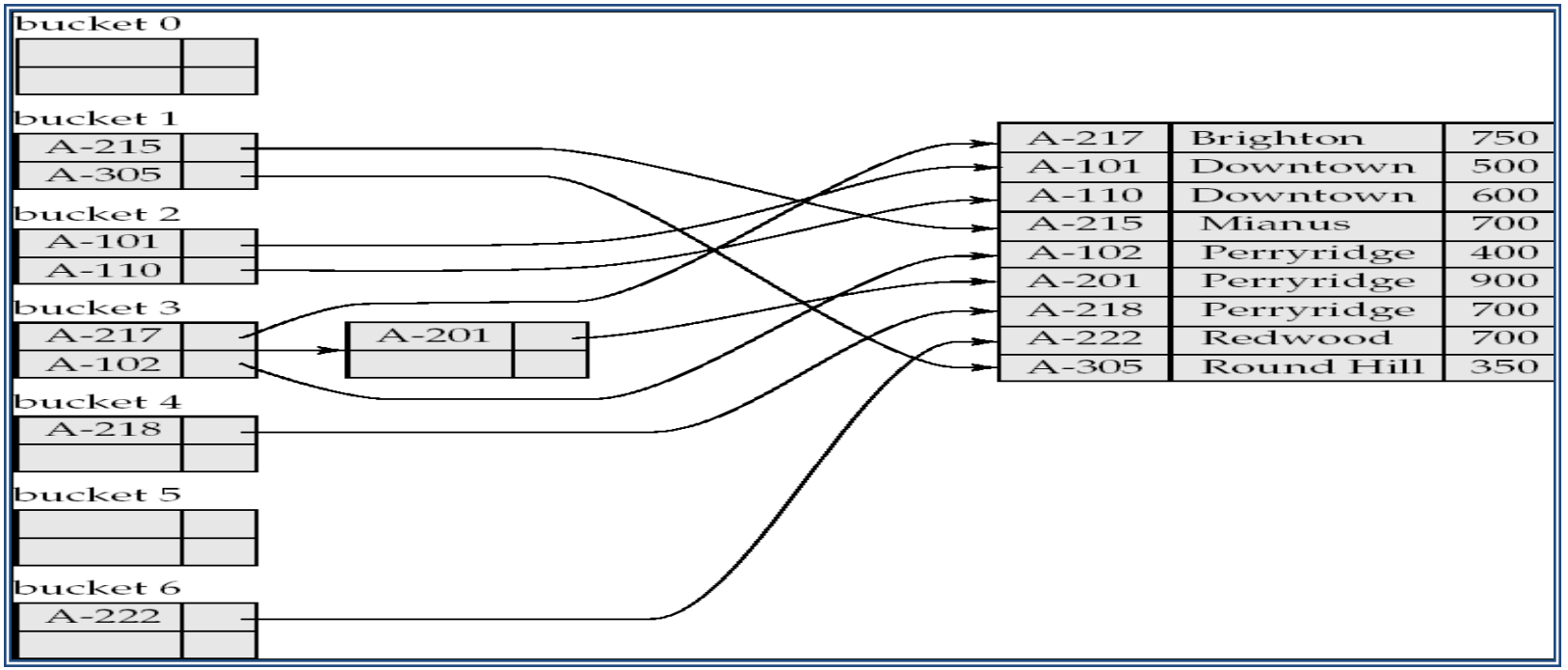
- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records. This can occur due to two reasons:
    - multiple records have same search-key value
    - chosen hash function produces non-uniform distribution of key values
- Although the probability of bucket overflow can be reduced, it cannot be eliminated; it is handled by using *overflow buckets.*
- Overflow chaining – the overflow buckets of a given bucket are chained together in a linked list.
  - Also called *closed hashing*
- Linear Probing - does not use overflow buckets, is not suitable for database applications.
  - Also called *open hashing*

Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation.
- A hash index organizes the search keys, with their associated record pointers, into a hash file structure.
- Strictly speaking, hash indices are always secondary indices
  - if the file itself is organized using hashing, a separate primary hash index on it using the same search-key is unnecessary.
  - However, we use the term hash index to refer to both secondary index structures and hash organized files.
- We construct a hash index as follows. We apply a hash function on a search key to identify a bucket, and store the key and its associated pointers in the bucket (or in overflow buckets).

Hash Indices(contd…)

- Figure below shows a secondary hash index on the *account file, for the search key account-number. The hash function in the figure* computes the sum of the digits of the account number modulo 7. The hash index has seven buckets, each of size 2. One of the buckets has three keys mapped to it, so it has an overflow bucket





The first approach to map the database to the file is to use the several files and store only one fixed length record in any given file. An alternative approach is to structure our files so that we can contain multiple lengths for records.

Files of fixed length records are easier to implement than the files of variable length records.

Objective of File Organization:

1. It contains an optimal selection of records, i.e., records can be selected as fast as possible.
2. To perform insert, delete or update transaction on the records should be quick and easy.
3. The duplicate records cannot be induced as a result of insert, update or delete.
4. For the minimal cost of storage, records should be stored efficiently.

Types of File Organization:

File organization contains various methods. These particular methods have pros and cons on the basis of access or selection. In the file organization, the programmer decides the best-suited file organization method according to his requirement.

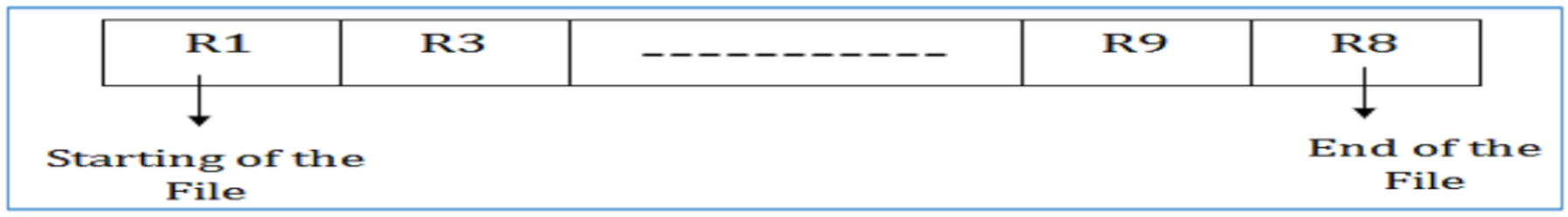
1.Sequential File Organization:

This method is the easiest method for file organization. In this method, files are stored sequentially. This method can be implemented in two ways:

a. Pile File Method:

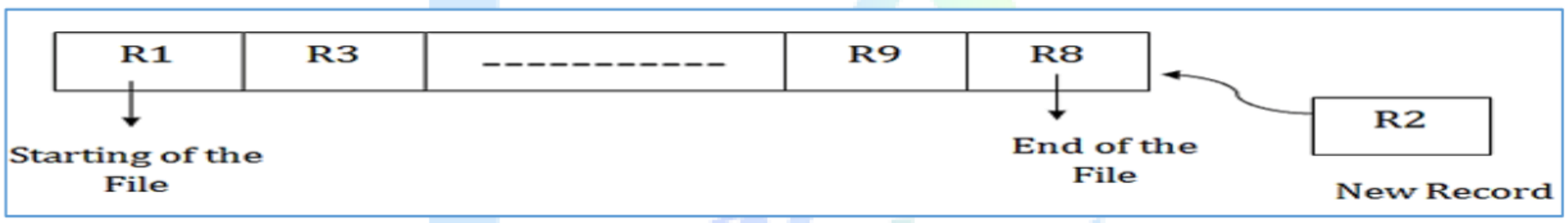
It is a quite simple method. In this method, we store the record in a sequence, i.e., one after another. Here, the record will be inserted in the order in which they are inserted into tables.

In case of updating or deleting of any record, the record will be searched in the memory blocks. When it is found, then it will be marked for deleting, and the new record is inserted.



Insertion of the New Record:

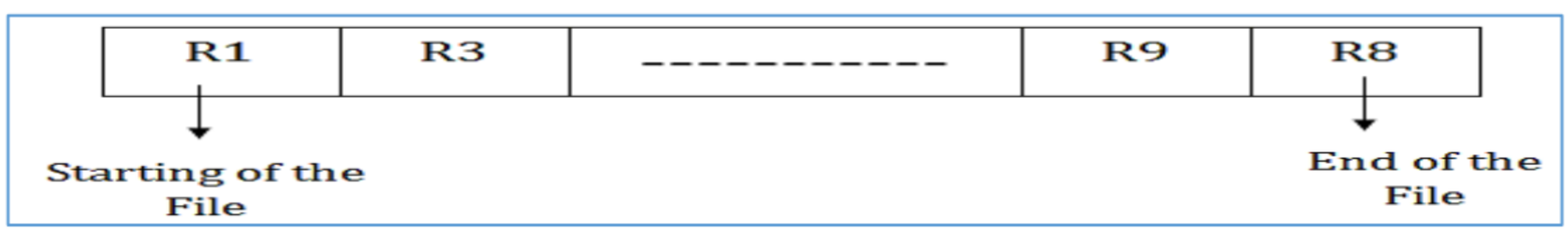
Suppose we have four records R1, R3 and so on upto R9 and R8 in a sequence. Hence, records are nothing but a row in the table. Suppose we want to insert a new record R2 in the sequence, then it will be placed at the end of the file. Here, records are nothing but a row in any table.



b. Sorted File Method:

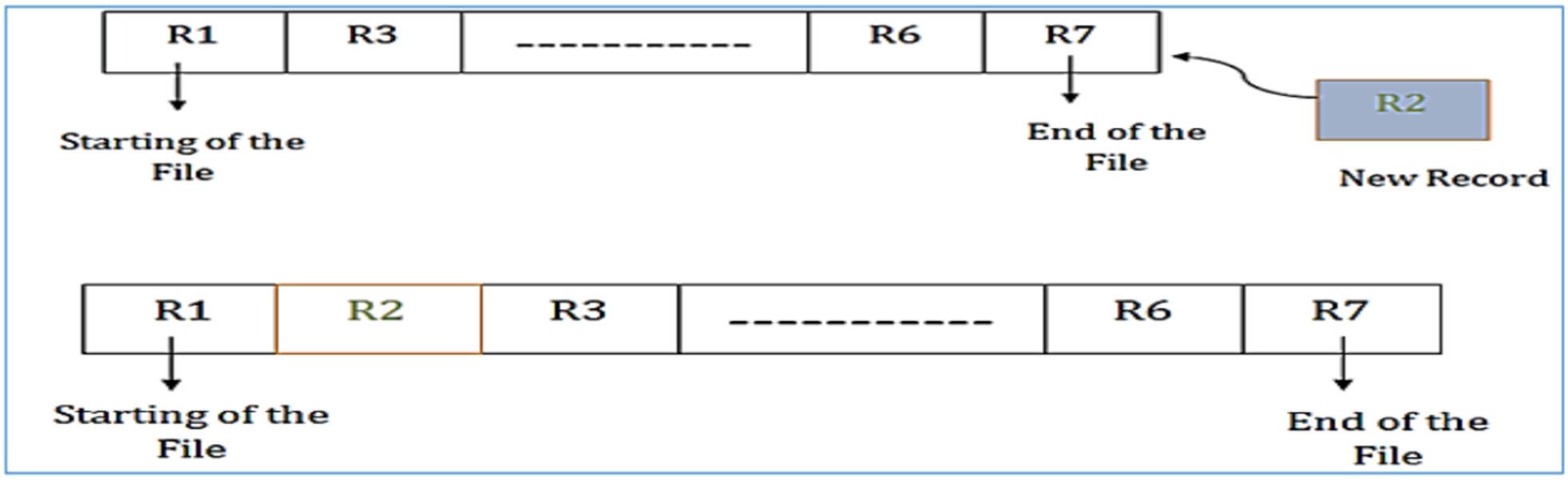
In this method, the new record is always inserted at the file's end, and then it will sort the sequence in ascending or descending order. Sorting of records is based on any primary key or any other key.

In the case of modification of any record, it will update the record and then sort the file, and lastly, the updated record is placed in the right place.



Insertion of the New Record:

Suppose there is a preexisting sorted sequence of four records R1, R3 and so on upto R6 and R7. Suppose a new record R2 has to be inserted in the sequence, then it will be inserted at the end of the file, and then it will sort the sequence.



Pros of Sequential File Organization:

a. It contains a fast and efficient method for the huge amount of data.

b. In this method, files can be easily stored in cheaper storage mechanism like magnetic tapes.

c. It is simple in design. It requires no much effort to store the data.

d. This method is used when most of the records have to be accessed like grade calculation of a student, generating the salary slip, etc.

e. This method is used for report generation or statistical calculations.

Cons of Sequential File Organization:

a. It will waste time as we cannot jump on a particular record that is required but we have to move sequentially which takes our time.

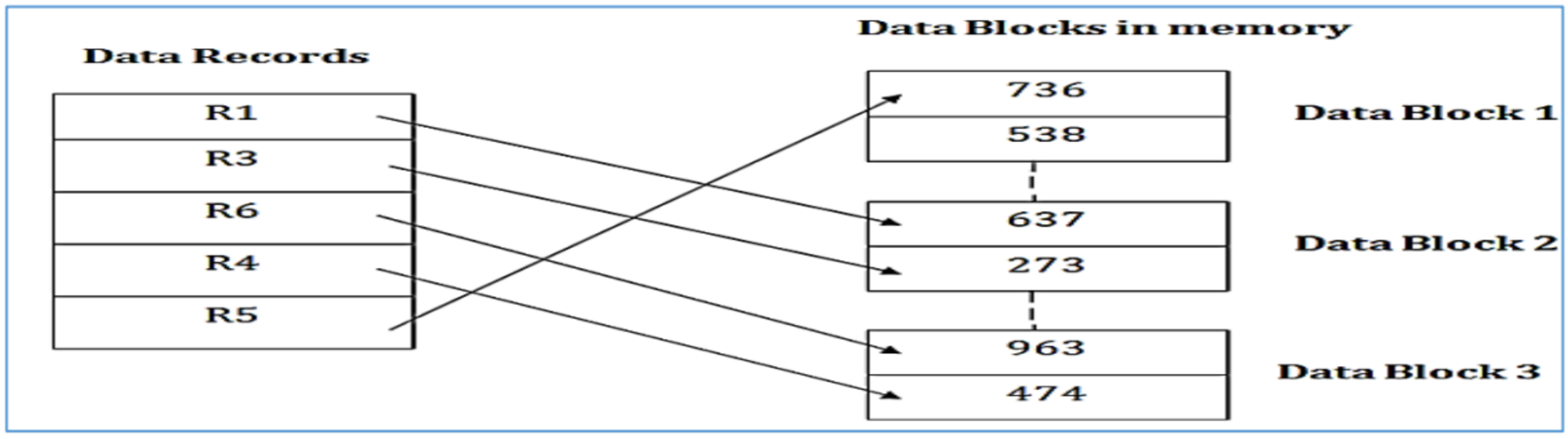
b. Sorted file method takes more time and space for sorting the records.

2.Heap File Organization:

It is the simplest and most basic type of organization. It works with data blocks. In heap file organization, the records are inserted at the file's end. When the records are inserted, it doesn't require the sorting and ordering of records.

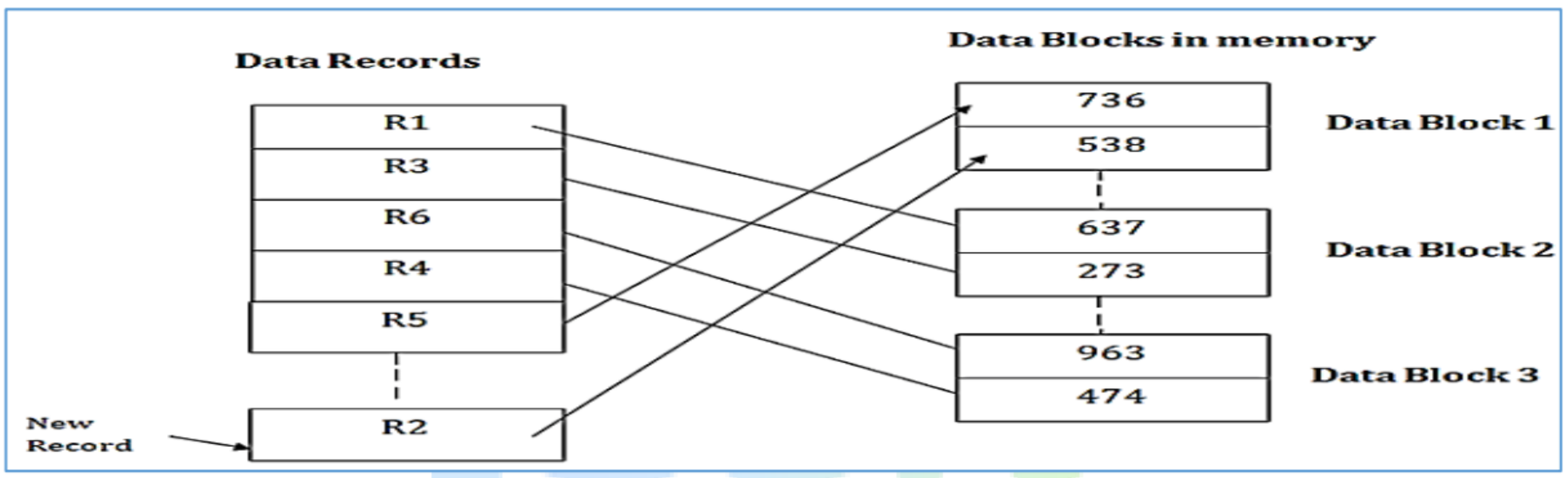
When the data block is full, the new record is stored in some other block. This new data block need not to be the very next data block, but it can select any data block in the memory to store new records. The heap file is also known as an unordered file.

In the file, every record has a unique id, and every page in a file is of the same size. It is the DBMS responsibility to store and manage the new records.



Insertion of a New Record:

Suppose we have five records R1, R3, R6, R4 and R5 in a heap and suppose we want to insert a new record R2 in a heap. If the data block 3 is full then it will be inserted in any of the database selected by the DBMS, let's say data block 1.



If we want to search, update or delete the data in heap file organization, then we need to traverse the data from staring of the file till we get the requested record.

If the database is very large then searching, updating or deleting of record will be time-consuming because there is no sorting or ordering of records. In the heap file organization, we need to check all the data until we get the requested record.

Pros of Heap File Organization:

a. It is a very good method of file organization for bulk insertion. If there is a large number of data which needs to load into the database at a time, then this method is best suited.
b. In case of a small database, fetching and retrieving of records is faster than the sequential record.

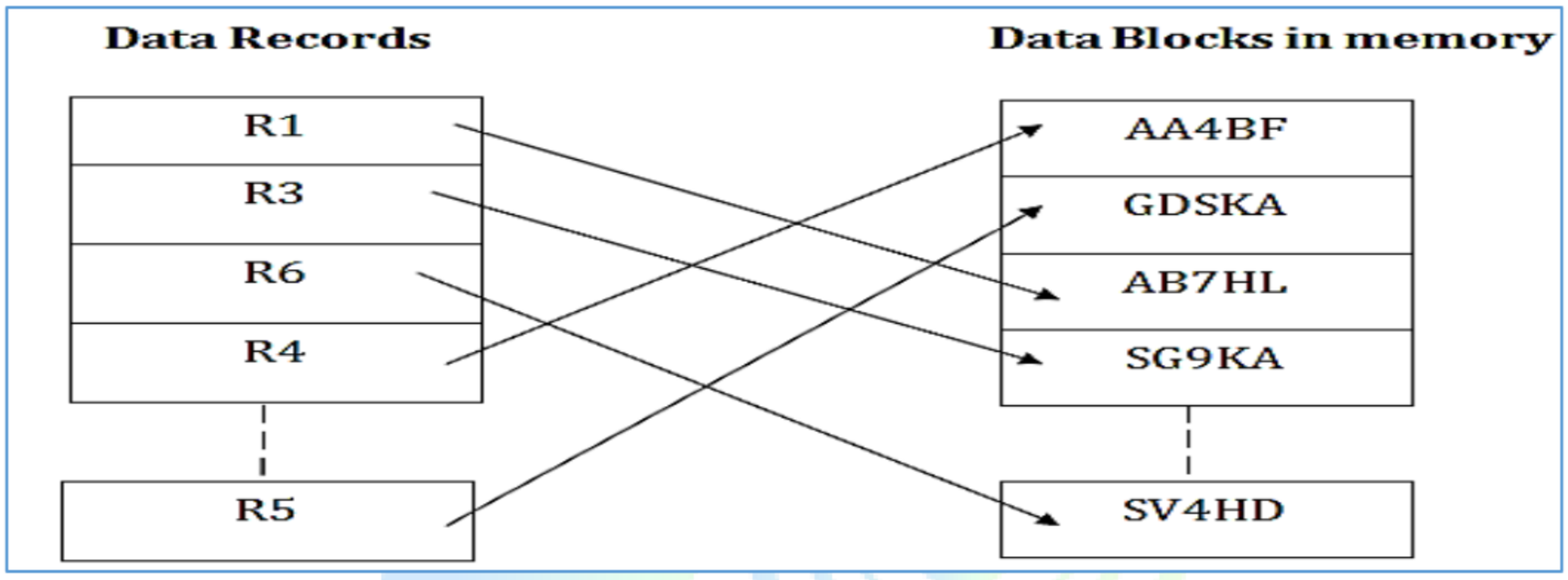
Cons of Heap File Organization:

a. This method is inefficient for the large database because it takes time to search or modify the record.

b. This method is inefficient for large databases.

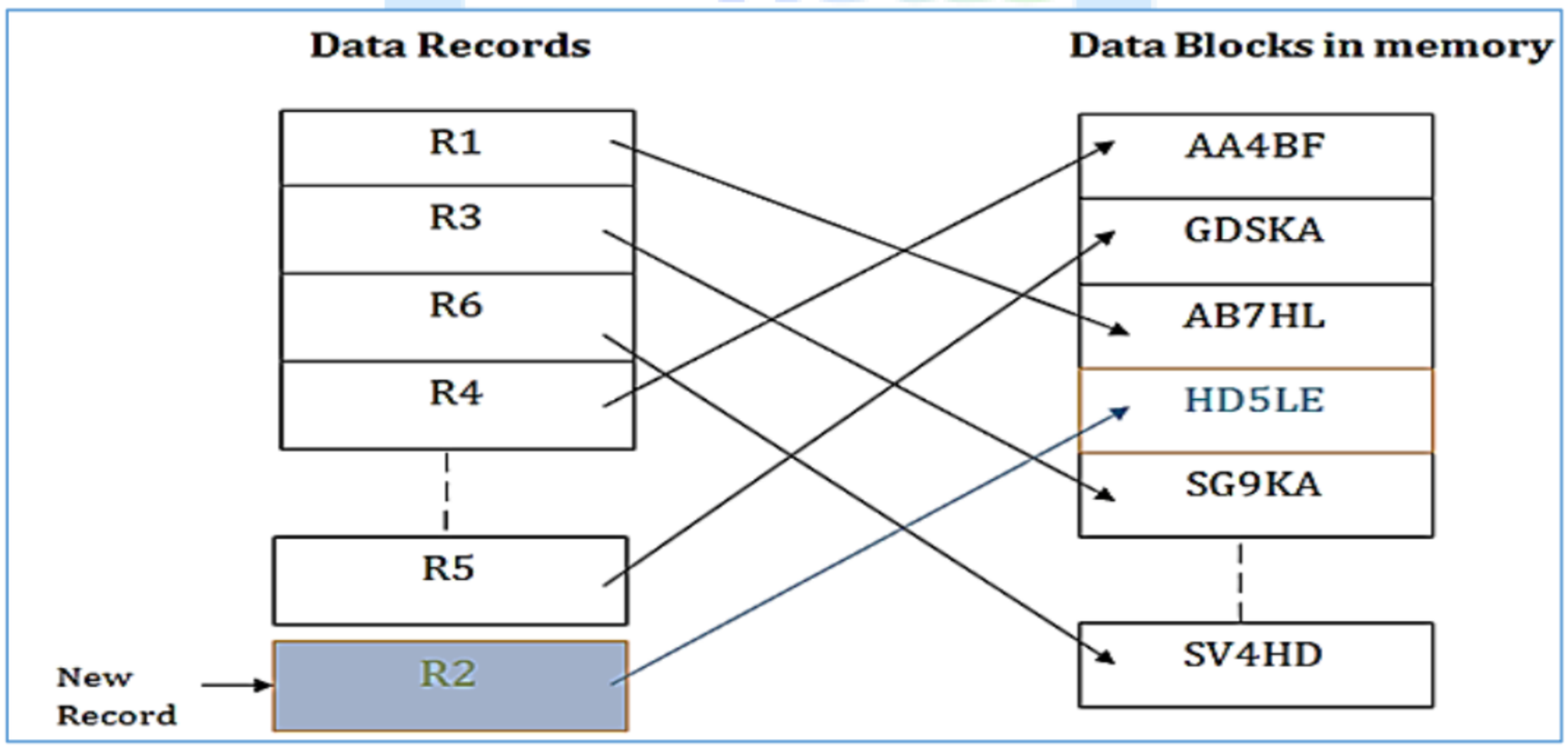
3.Hash File Organization:

Hash File Organization uses the computation of hash function on some fields of the records. The hash function's output determines the location of disk block where the records are to be placed.



When a record has to be received using the hash key columns, then the address is generated, and the whole record is retrieved using that address. In the same way, when a new record has to be inserted, then the address is generated using the hash key and record is directly inserted. The same process is applied in the case of delete and update.

In this method, there is no effort for searching and sorting the entire file. In this method, each record will be stored randomly in the memory.

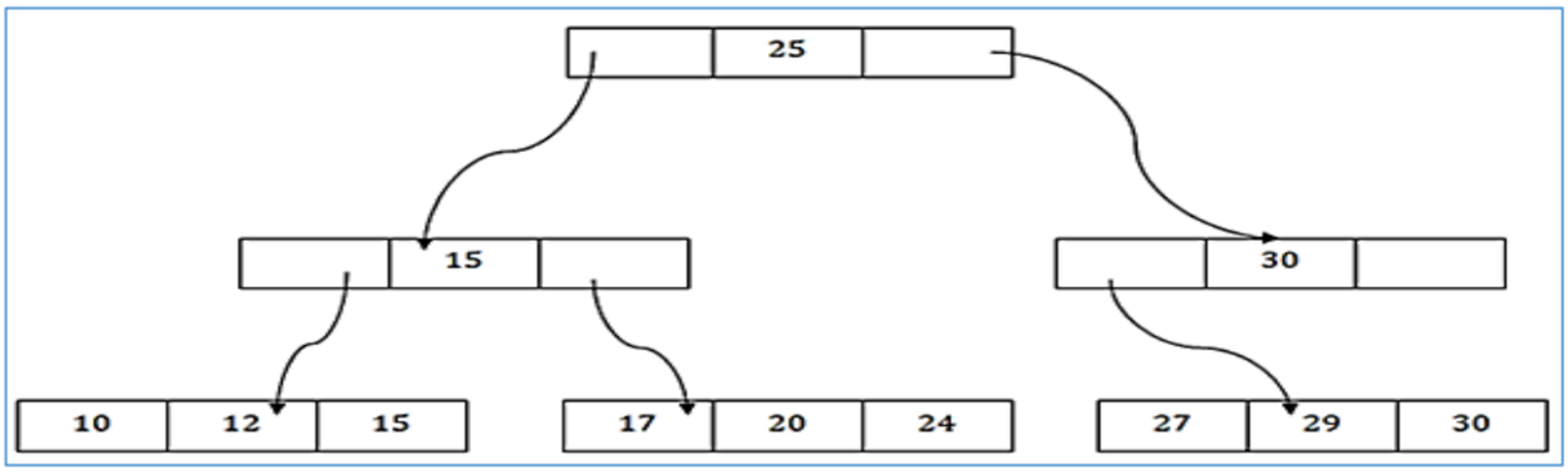


4.B+ File Organization:

B+ tree file organization is the advanced method of an indexed sequential access method. It uses a tree-like structure to store records in File. It uses the same concept of key-index

where the primary key is used to sort the records. For each primary key, the value of the index is generated and mapped with the record.

The B+ tree is similar to a binary search tree (BST), but it can have more than two children. In this method, all the records are stored only at the leaf node. Intermediate nodes act as a pointer to the leaf nodes. They do not contain any records.



The Above B+ Tree Shows That:

a. There is one root node of the tree, i.e., 25.

b. There is an intermediary layer with nodes. They do not store the actual record. They have only pointers to the leaf node.

c. The nodes to the left of the root node contain the prior value of the root and nodes to the right contain next value of the root, i.e., 15 and 30 respectively.

d. There is only one leaf node which has only values, i.e., 10, 12, 17, 20, 24, 27 and 29.

e. Searching for any record is easier as all the leaf nodes are balanced.

f. In this method, searching any record can be traversed through the single path and accessed easily.

Pros of B+ Tree File Organization:

a. In this method, searching becomes very easy as all the records are stored only in the leaf nodes and sorted the sequential linked list.

b. Traversing through the tree structure is easier and faster.

c. The size of the B+ tree has no restrictions, so the number of records can increase or decrease and the B+ tree structure can also grow or shrink.

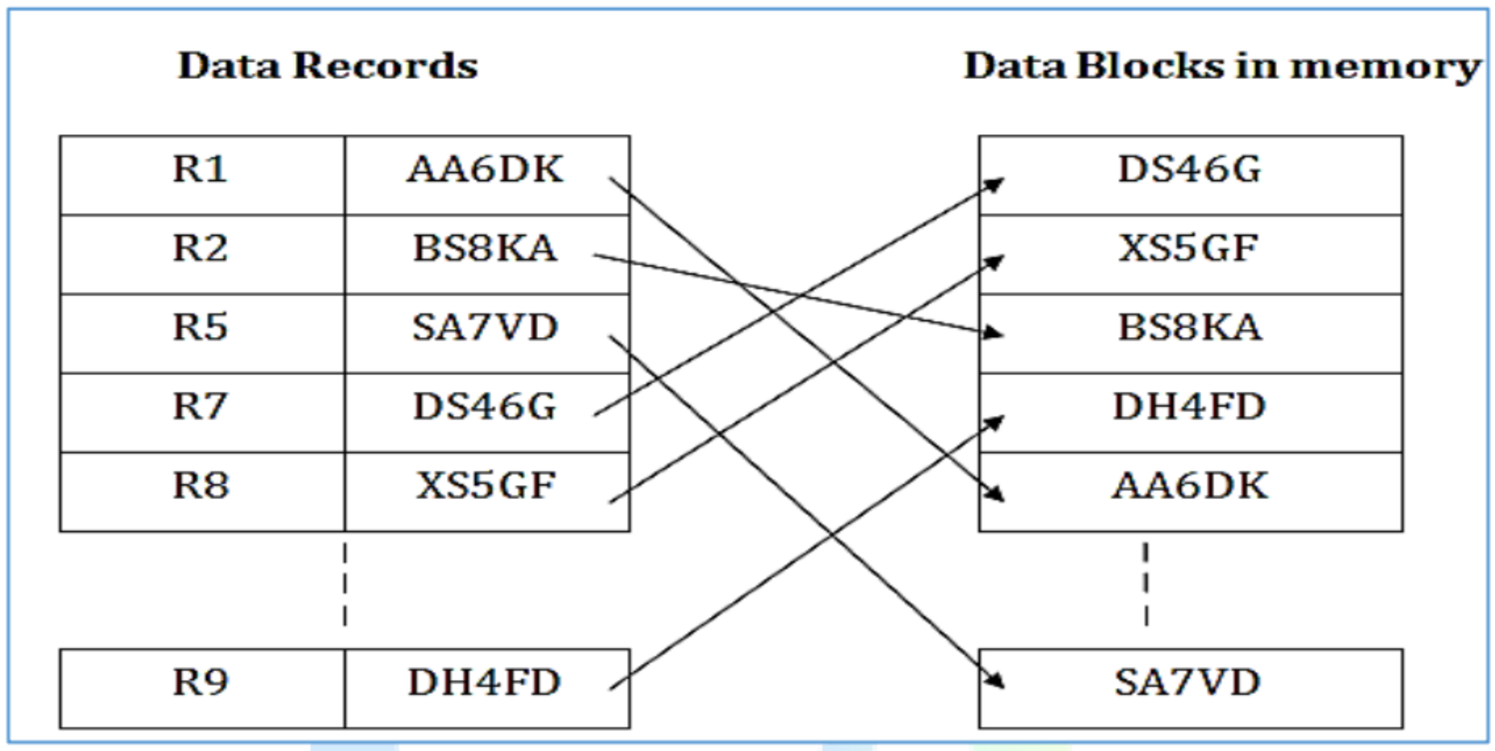
d. It is a balanced tree structure, and any insert/update/delete does not affect the performance of tree.

Cons of B+ Tree File Organization:

a. This method is inefficient for the static method.

5.Indexed Sequential Access Method (ISAM):

ISAM method is an advanced sequential file organization. In this method, records are stored in the file using the primary key. An index value is generated for each primary key and mapped with the record. This index contains the address of the record in the file.



If any record has to be retrieved based on its index value, then the address of the data block is fetched and the record is retrieved from the memory.

Pros of ISAM:

a. In this method, each record has the address of its data block, searching a record in a huge database is quick and easy.

b. This method supports range retrieval and partial retrieval of records. Since the index is based on the primary key values, we can retrieve the data for the given range of value. In the same way, the partial value can also be easily searched, i.e., the student name starting with 'JA' can be easily searched.

Cons of ISAM:

a. This method requires extra space in the disk to store the index value.

b. When the new records are inserted, then these files have to be reconstructed to maintain the sequence.

c. When the record is deleted, then the space used by it needs to be released. Otherwise, the performance of the database will slow down.

6. Cluster File Organization:

When the two or more records are stored in the same file, it is known as clusters. These files will have two or more tables in the same data block, and key attributes which are used to map these tables together are stored only once.

This method reduces the cost of searching for various records in different files. The cluster file organization is used when there is a frequent need for joining the tables with the same condition.

These joins will give only a few records from both tables. In the given example, we are retrieving the record for only particular departments. This method can't be used to retrieve the record for the entire department.

EMPLOYEE

EMP_ID	EMP_NAME	ADDRESS	DEP_ID
1	John	Delhi	14
2	Robert	Gujarat	12
3	David	Mumbai	15
4	Amelia	Meerut	11
5	Kristen	Noida	14
6	Jackson	Delhi	13
7	Amy	Bihar	10
8	Sonoo	UP	12

DEPARTMENT

DEP_ID	DEP_NAME
10	Math
11	English
12	Java
13	Physics
14	Civil
15	Chemistry

Cluster Key

DEP_ID	DEP_NAME	EMP_ID	EMP_NAME	ADDRESS
10	Math	7	Amy	Bihar
11	English	4	Amelia	Meerut
12	Java	2	Robert	Gujarat
12		8	Sonoo	UP
13	Physics	6	Jackson	Delhi
14	Civil	1	John	Delhi
14		5	Kristen	Noida
15	Chemistry	3	David	Mumbai

In this method, we can directly insert, update or delete any record. Data is sorted based on the key with which searching is done. Cluster key is a type of key with which joining of the table is performed.

Types of Cluster File Organization:

1. Indexed Clusters:

In indexed cluster, records are grouped based on the cluster key and stored together. The above EMPLOYEE and DEPARTMENT relationship is an example of an indexed cluster. Here, all the records are grouped based on the cluster key- DEP_ID and all the records are grouped.

2. Hash Clusters:

It is similar to the indexed cluster. In hash cluster, instead of storing the records based on the cluster key, we generate the value of the hash key for the cluster key and store the records with the same hash key value.

Pros of Cluster File Organization:

a. The cluster file organization is used when there is a frequent request for joining the tables with same joining condition.
b. It provides the efficient result when there is a 1:M mapping between the tables.

Cons of Cluster File Organization:

a. This method has the low performance for the very large database.
b. If there is any change in joining condition, then this method cannot use. If we change the condition of joining then traversing the file takes a lot of time.
c. This method is not suitable for a table with a 1:1 condition.

Indexing in DBMS:

Indexing is used to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed. The index is a type of data structure. It is used to locate and access the data in a database table quickly.

Index Structure:

Indexes can be created using some database columns.

Search key	Data Reference

Fig: Structure of Index

The first column of the database is the search key that contains a copy of the primary key or candidate key of the table. The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily.

The second column of the database is the data reference. It contains a set of pointers holding the address of the disk block where the value of the particular key can be found.

Indexing Methods:

1. Ordered Indices:

The indices are usually sorted to make searching faster. The indices which are sorted are known as ordered indices.

Example:

Suppose we have an employee table with thousands of record and each of which is 10 bytes long. If their IDs start with 1, 2, 3....and so on and we have to search student with ID-543.

1. In the case of a database with no index, we have to search the disk block from starting till it reaches 543. The DBMS will read the record after reading 543*10=5430 bytes.
2. In the case of an index, we will search using indexes and the DBMS will read the record after reading 542*2= 1084 bytes which are very less compared to the previous case.

2. Primary Index:

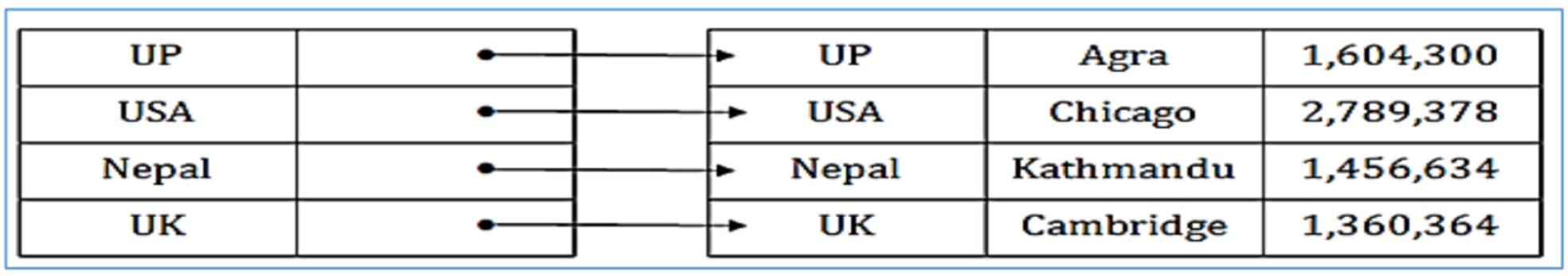
If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.

As primary keys are stored in sorted order, the performance of the searching operation is quite efficient. The primary index can be classified into two types: Dense index and Sparse index.

3. Dense Index:

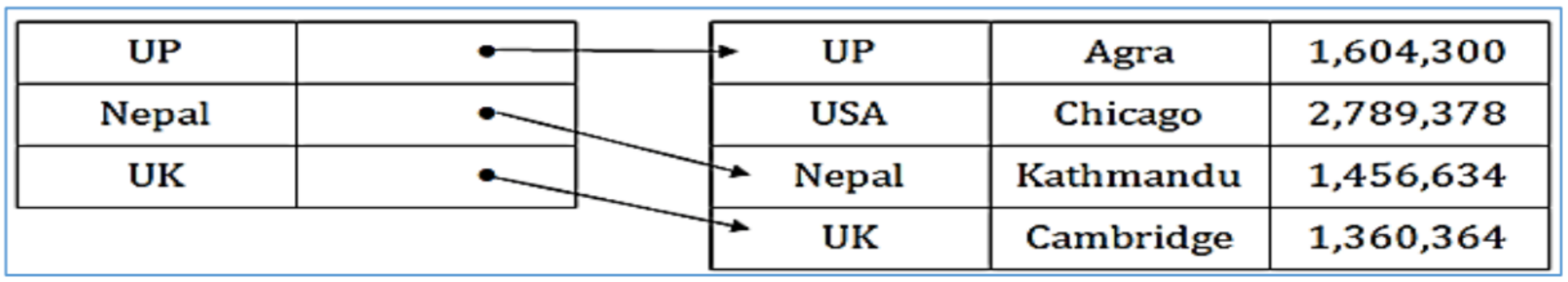
The dense index contains an index record for every search key value in the data file. It makes searching faster. In this, the number of records in the index table is same as the number of records in the main table.

It needs more space to store index record itself. The index records have the search key and a pointer to the actual record on the disk.



4. Sparse Index:

In the data file, index record appears only for a few items. Each item points to a block. In this, instead of pointing to each record in the main table, the index points to the records in the main table in a gap.



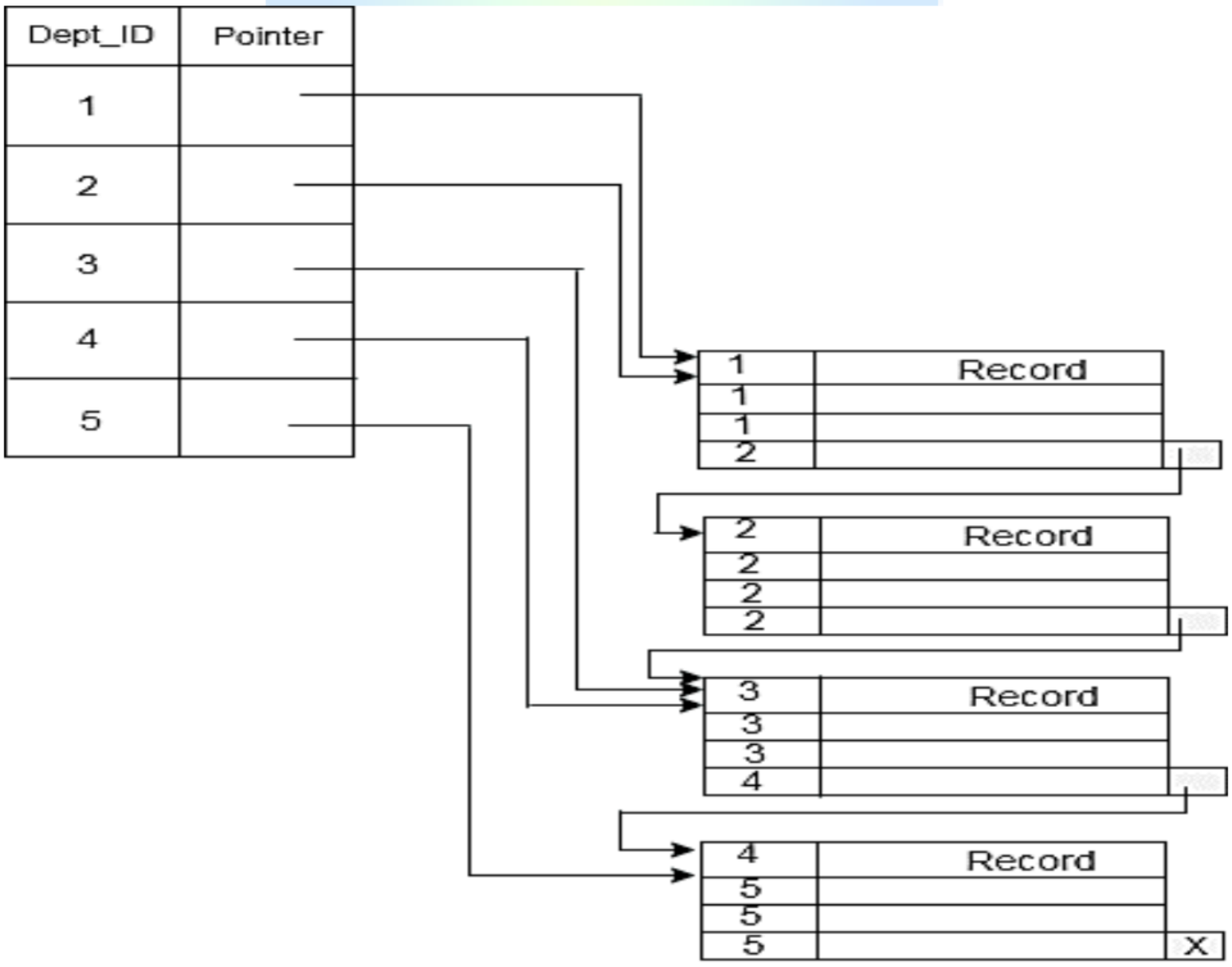
5. Clustering Index

A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.

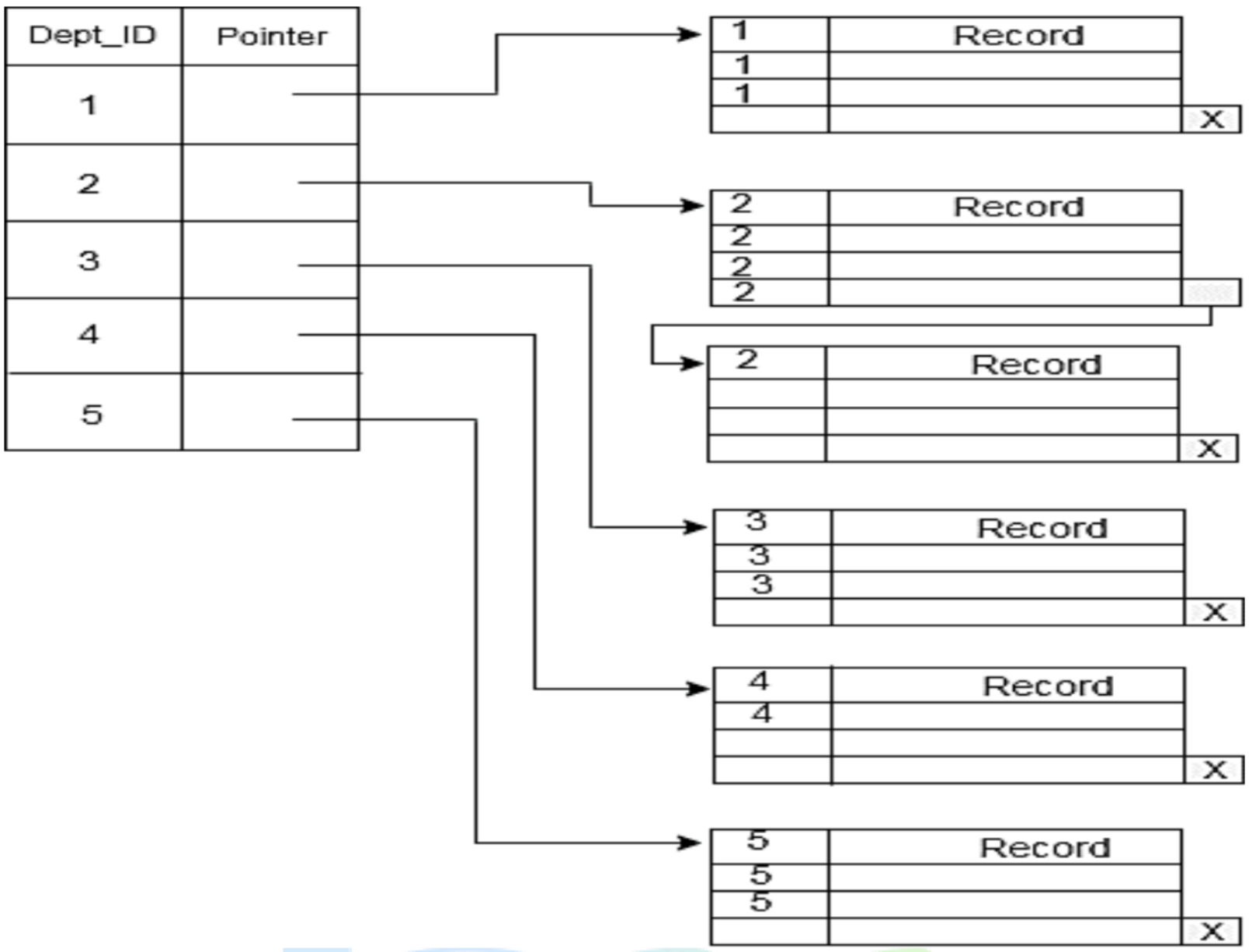
In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.

The records which have similar characteristics are grouped, and indexes are created for these group.

Example: suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_Id is a non-unique key.



The previous schema is little confusing because one disk block is shared by records which belong to the different cluster. If we use separate disk block for separate clusters, then it is called better technique.



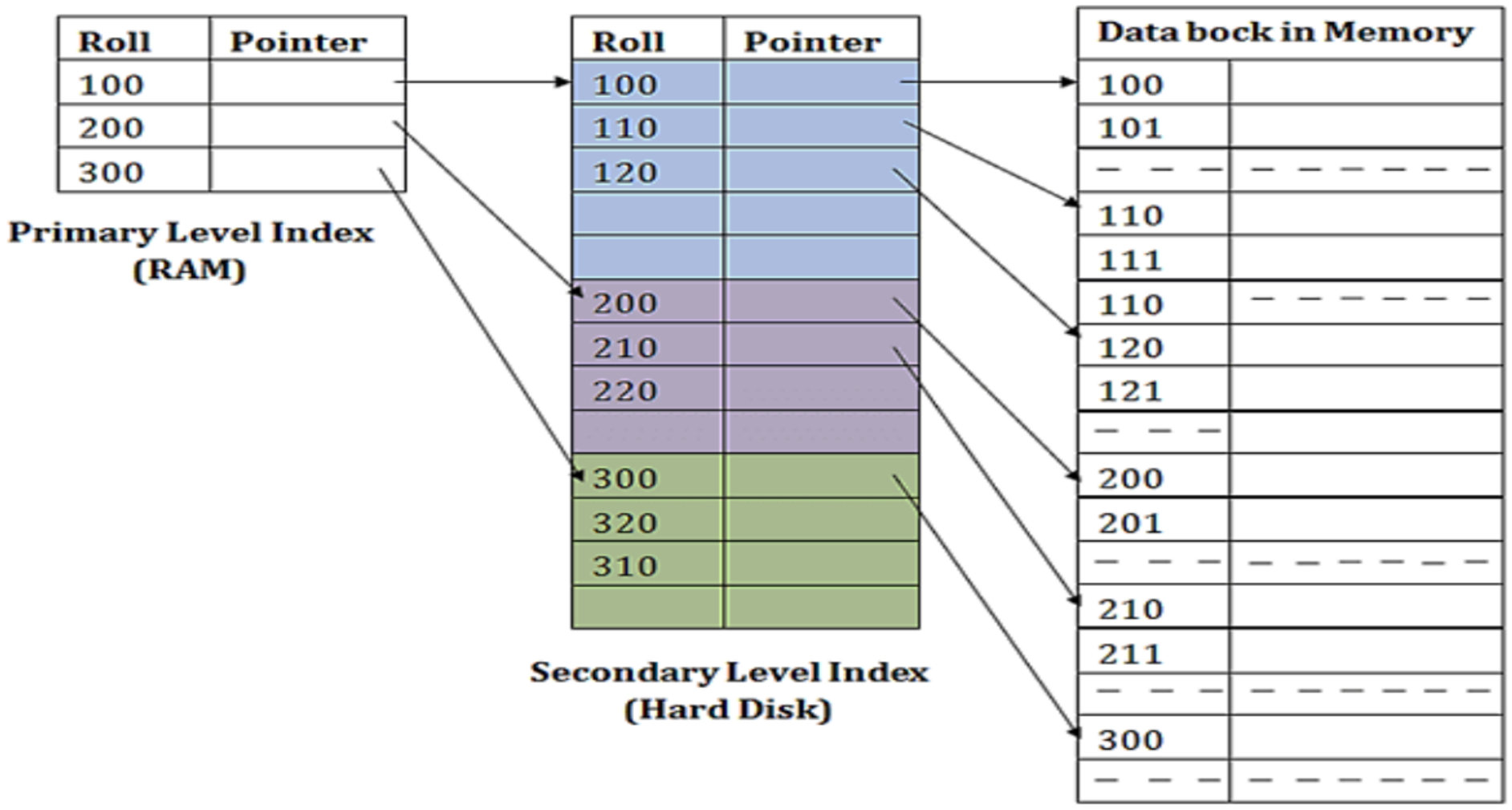
6. Secondary Index:

In the sparse indexing, as the size of the table grows, the size of mapping also grows. These mappings are usually kept in the primary memory so that address fetch should be faster.

Then the secondary memory searches the actual data based on the address got from mapping. If the mapping size grows then fetching the address itself becomes slower. In this case, the sparse index will not be efficient. To overcome this problem, secondary indexing is introduced.

In secondary indexing, to reduce the size of mapping, another level of indexing is introduced. In this method, the huge range for the columns is selected initially so that the mapping size of the first level becomes small.

Then each range is further divided into smaller ranges. The mapping of the first level is stored in the primary memory, so that address fetch is faster. The mapping of the second level and actual data are stored in the secondary memory (hard disk).



For Example:

1. If we want to find the record of roll 111 in the diagram, then it will search the highest entry which is smaller than or equal to 111 in the first level index. It will get 100 at this level.
2. Then in the second index level, again it does max (111) <= 111 and gets 110. Now using the address 110, it goes to the data block and starts searching each record till it gets 111.
3. This is how a search is performed in this method. Inserting, updating or deleting is also done in the same manner.

Type of Indexes:

1. Function-Based Indexes:

- A function-based index computes the values of expression which are present in one or more column and stored in the table.
- The expression can be an arithmetic expression or SQL function.
- A function-based index cannot contain null value.

Example: CREATE INDEX Sample _idx on table_(a+c*(b+d));

2. Bitmap Indexes:

- Bitmap index is used to work with well for low-cardinal (refers to columns few unique values) columns in tables. **For example:** Boolean data which has only two values true or false.

- Bitmap indexes are very useful in data ware house applications for joining the large fact tables.

3. Domain Indexes:

- Domain index is used to create index type schema object and an application specific index. It is used for indexing data in application specific domain.

4. Clusters:

- Clustering in DBMS is design for high availability of data. Clustering is applied on tables which are repeatedly used by the user. **For example:** When there are many employees in the department, we can create index of non-unique key, such that Dept-id. With this, all employees belonging to the same department are consider to be within a same cluster.
- Clustering can improve the performance of the system.

5. Indexed Sequential Access Method (ISAM):

- ISAM was developed by IBM for mainframe computers but the term is used in several concepts.
- In DBMS, ISAM is used to access data in sequentially (sequence in which data is entered) or randomly (with an index).